

Counterexample Tests for Computational Research Evaluation

Kyunghan Bae
UNICUP COMPANY, Republic of Korea
ceo@unicupcompany.com

Abstract

Research evaluators can return plausible numbers while losing the relationship between an array position and the entity it represents. This letter documents three implementation counterexamples in fixed versions of public research code. The cases concern patient-level survival targets, channel layout in structural similarity, and model-aligned ranks. Source inspection connects each fixture to the released caller rather than assuming an unsupported input contract. Executions of pinned functions and a PyTorch cross-check support the numerical findings. Correcting the model-rank conversion restores identity-aligned results for all six permutations of a three-model fixture. Survival comparisons check normalized inputs against a direct mean-squared-error oracle, including controls for input shapes and observation counts. These results provide local diagnoses and executable repair checks; measuring their effect on original empirical tables requires the original evaluation material.

Keywords: research software; evaluation metrics; computational reproducibility; metamorphic testing; input contracts

1. Evaluation errors and interpretable tests

A floating-point result is easy to store in an experimental table. It is harder to establish that the calculation still represents the intended patient, image channel, or model after data loaders and array operations have changed its representation. An extra dimension may be accepted by a numerical library, and a sorting function may return valid indices that are later interpreted as a different quantity.

Metamorphic testing checks relationships between executions when a complete numerical oracle is unavailable. It has already been applied to scientific software [1]. The contribution here is a set of implementation diagnoses linked to actual callers, together with small comparisons that test a repair. It is not a new general testing algorithm. Each case specifies the entity being preserved, the relevant source contract, the operation that changes its meaning, and a comparison with an expected result.

Three repositories were selected from earlier investigations because their failures could be represented by compact synthetic fixtures. This purposive selection supports the reported mechanisms and repair comparisons. It does not support an estimate of defect prevalence or detection accuracy. All reported observations are generated from synthetic inputs; the original empirical tables were not recalculated.

2. Sources, execution and comparison contracts

The fixed source versions are Dynamic-DeepHit at commit b2e208f65233405079a462b703a94556ad30d1f4 [2], DBAE at 533f5648a3a1ca419c402c656e305536267c14c8 [3], and ArxivRoll at 709b0c5738b1710a4ac6988d70b32ffb1d21458b [4]. The supplement identifies raw-file URLs, SHA-256 values, fixtures, execution types, and numerical outputs. Runners download and hash-check the relevant sources before extracting selected functions. Source bodies from the original repositories are not redistributed in the fixture archive.

For survival evaluation and ranking, extracted source functions are executed directly. The primary image comparison implements the reviewed arithmetic in NumPy; a separate comparison executes the four original PyTorch functions. These execution types are recorded separately. They establish behavior of the selected operations rather than execution of a complete original framework.

A comparison has three requirements. First, the fixture must satisfy a contract that the released caller can actually produce. Second, the expected relationship must follow the evaluated statistic. Third, a proposed change must restore that relationship without erasing meaningful dimensions. The controls below therefore include direct targets, restored model identities, and explicit channel-layout conversion. They do not treat arbitrary transformations as invariances.

3. Patient targets in survival evaluation

The Dynamic-DeepHit data loader uses a column selection for survival times: `pat_info[:, [1]]` in `import_data.py`, line 140. Its main evaluation code passes column-shaped test times together with a one-dimensional event vector to the examined evaluators (`main.py`, lines 293–294). The dimensionality tested here consequently matches the released caller.

For two uncensored patients, let times be 1 and 3, the event indicators be (1, 1), and the horizon be 2. The patient-level target is (1, 0). A perfect prediction must have mean squared error 0, while its complement has error 1. With times of shape (2, 1), the examined Brier function multiplies a column time indicator by an event vector of shape (2,). Broadcasting creates a (2, 2) target. Perfect and reversed predictions both return 0.5. Supplying the same times as a one-dimensional vector returns 0 and 1. In the same fixture, the examined cause-specific concordance changes from 0.5 to 1 for the perfect prediction.

The repair comparison normalizes a supported single-column time vector to a one-dimensional vector and checks the observation counts of predictions, times, and events. It rejects multi-column times instead of flattening them, because flattening could convert distinct measurements into invented patients. The Brier comparisons use an independent oracle, the arithmetic mean of squared differences between each prediction and its uncensored binary target. Perfect, reversed, and constant predictions provide different controls rather than a single successful example.

The mechanism can also be expressed algebraically. For these uncensored one-event fixtures, with all event indicators equal to one, let y be the binary target and p the prediction. The broadcast calculation is $\text{mean}(p^2) - 2 \text{mean}(p) \text{mean}(y) + \text{mean}(y)$, whereas the patient-paired calculation is $\text{mean}(p^2) - 2 \text{mean}(p y) + \text{mean}(y)$. Their difference is twice the population covariance of p and y . Broadcasting replaces the patient-specific association with all cross-patient combinations. Constant predictions have zero covariance and therefore provide a positive control even when the same shape error is present.

The extended comparison uses sample sizes 2, 3, 5, and 10, two integer horizons, and perfect, reversed, and constant predictions: 24 designed conditions. For two patients, the distinct horizons produce the same target split. The original column-input calculation differs from the oracle in 16 conditions. The normalized comparison agrees in all 24, including all eight constant-prediction controls. Multi-column times and a mismatched observation count are both rejected. These counts describe the fixture grid, not an observed error rate in patient data or a repository sample.

The experiment uses the unweighted Brier function and uncensored fixtures. A complete survival evaluation would also require examining censoring, event types, time-dependent weights, and the exact version used to produce a published table. Input normalization alone is not a validation of those estimators.

4. Channel layout in image similarity

The DBAE reconstruction evaluator calls SSIM directly on its stored image arrays (`eval_reconstruction.py`, line 45), while the LPIPS call in the same code converts them with `permute(0, 3, 1, 2)` (line 49). The repository README names this evaluator in its Table 2 reproduction instructions. The inconsistent treatment of layout is therefore part of the released evaluation route.

The synthetic RGB images have shape (1, 128, 128, 3), representing batch, height, width, and channels (NHWC). The examined similarity calculation instead treats the second axis as channels, consistent with NCHW. Direct NHWC input is interpreted as 128 channels on a spatial domain of 128 by 3. A uniform random original image and a comparison image containing a shifted band of rows make the numerical consequence observable.

The NumPy comparison uses the reviewed 11 by 11 Gaussian window, standard deviation 1.5, zero padding, and per-channel arithmetic. Direct NHWC input yields SSIM 0.8221873479020841. Explicit conversion of both inputs to NCHW yields 0.6235561674821463. Interchanging height and width gives 0.8246057307192767 in the former interpretation and leaves the latter unchanged. This symmetry is justified by this fixture and isotropic window; it is not a requirement for direction-dependent image processing.

As a translation check, the pinned gaussian, `create_window`, `_ssim`, and `ssim` functions were executed with PyTorch 2.14.1+cpu on all four layouts. The maximum absolute difference from NumPy was 8.47e-8, below the specified tolerance of 1e-6. The original window is first created in float32 and then converted to the image dtype, which explains why agreement is approximate rather than bitwise.

These comparisons isolate an axis interpretation and its correction. Original reconstruction files, trained models, LPIPS results, and changes in the original model ranking remain unmeasured. The direct SSIM discrepancy cannot be substituted for an effect size on the published experiment.

5. Model identity in relative ranks

ArxivRoll uses sorting indices in its relative-rank calculation (`rs.py`, lines 50–52). Its caller later attaches result element *i* to model *i* (lines 383–385). This requires a rank aligned with each original model, while a sorting permutation instead identifies the model occupying each sorted position. The examined helper also reverses benchmark columns with `[:, ::-1]`; that operation does not convert sorted model indices to model-aligned ranks.

The fixture has three models and two benchmarks, all with distinct scores. Public scores are $A=(0.9, 0.6)$, $B=(0.7, 0.8)$, $C=(0.5, 0.4)$; private scores are $A=(0.5, 0.8)$, $B=(0.9, 0.4)$, $C=(0.7, 0.6)$. Equal unmatched components contribute zero. The examined code produces relative values (0.7, -0.7, 0). Swapping A and B in both matrices and restoring model identities produces (-0.7, 0.7, 0), although each model retains its input scores.

An extended comparison examines every permutation of these three rows. After restoring identities, five of the six original outputs differ from the original baseline. The same relative-score calculation is then executed with only its rank-conversion helper replaced. For the tie-free, higher-is-better fixture, model-aligned one-based ranks are obtained by the inverse sorting permutation, `argsort(argsort(-array, axis=0), axis=0) + 1`. This comparator assigns rank 1 to the largest score and retains the input benchmark-column order. All six repaired outputs agree after identity restoration: (-1/6, -1/6, 0.4).

This change diagnoses the model-alignment mechanism; it does not specify a policy for ties, missing values, lower-is-better metrics, or aggregates with different score directions. Those policies must be stated before the same comparison is applied to another benchmark collection. Invariance alone also does not establish that the relative-rank formula is the most appropriate scientific summary.

Table 1 summarises the implementation comparisons. In each row, the expected relationship is attached to a stated contract rather than to arbitrary input transformations.

Case and contract	Original comparison	Repair or comparison control	Execution evidence
Dynamic-DeepHit, uncensored one-event targets	Column times: perfect and reversed Brier both 0.5; 16/24 grid values differ from the oracle	Single-column normalization: 0/24 oracle differences; 8/8 constant controls agree; two unsupported inputs rejected	Extracted source function, independent paired-error oracle, analytic broadcast expression
DBAE, specified isotropic SSIM window	NHWC 0.822187; H/W exchange 0.824606	Explicit NCHW 0.623556 in both orientations	NumPy arithmetic and original PyTorch functions agree within 1e-6 on four layouts
ArxivRoll, tie-free higher-is-better scores	Five of six identity-restored row permutations differ from baseline	Correct model-aligned ranks: all six yield (-1/6, -1/6, 0.4) after identity restoration	Extracted source functions; only rank conversion changed

6. Interpretation and use

The three cases have different numerical oracles but a common representational requirement. A patient target must remain attached to that patient; a channel axis must remain a channel axis; a reported rank must remain attached to the model it describes. Linking a fixture to a loader and caller is essential: an evaluator can reject an unsupported input without being defective, and a transformation can intentionally change the estimand.

Repair comparisons strengthen a diagnosis because they test whether the specific operation identified as the cause explains the failing relationship. Positive controls prevent a report from equating any changed output with an improvement. A supported column-to-vector conversion restores uncensored mean squared error, explicit layout conversion restores the specified SSIM interpretation, and inverse permutation restores model-aligned rank outputs. Each conclusion is restricted to its contract and tested material.

The supplementary material includes a separate synthetic two-sample resampling illustration. It reconstructs a simplified linear estimator rather than the original R analysis. This methodological illustration is distinct from the three implementation cases in this letter. Quantifying a repair in that study would require the original estimator and its dependence or clustering design.

The present results support local implementation diagnoses and reusable regression comparisons. They leave three substantial research tasks open: broader input families and ranking policies, independent checks of repaired evaluators on real evaluation material, and effects on original reported scores or conclusions. A large-scale study of detection performance would additionally need a defined repository sampling procedure and suitable baselines. The documented fixtures can serve as starting points for those tasks.

Code and data

Executable fixtures, pinned-source records, and numerical outputs are available at <https://github.com/qorud02/research-evaluation-counterexamples>. The archive includes repair comparisons and caller-contract evidence as separate supplementary files. All experimental inputs are synthetic and reproducible. The archive distinguishes original implementation executions, translated arithmetic, repaired comparison operations, and the separate resampling illustration.

References

1. Lin X, Simon M, Niu N. Exploratory Metamorphic Testing for Scientific Software. *Computing in Science and Engineering*. 2020;22(2):78–87. <https://doi.org/10.1109/MCSE.2018.2880577>
2. Dynamic-DeepHit contributors. `utils_eval.py`, `import_data.py`, `main.py`. Commit `b2e208f65233405079a462b703a94556ad30d1f4`. <https://github.com/chl8856/Dynamic-DeepHit/tree/b2e208f65233405079a462b703a94556ad30d1f4>
3. DBAE contributors. `eval_reconstruction.py` and `README.md`. Commit `533f5648a3a1ca419c402c656e305536267c14c8`. <https://github.com/aailab-kaist/DBAE/tree/533f5648a3a1ca419c402c656e305536267c14c8>
4. ArxivRoll contributors. `rs.py`. Commit `709b0c5738b1710a4ac6988d70b32ffb1d21458b`. <https://github.com/liangzid/ArxivRoll/blob/709b0c5738b1710a4ac6988d70b32ffb1d21458b/rs.py>